Department of Electrical, Computer, & Biomedical Engineering
Faculty of Engineering & Architectural Science

Course Number	ELE 888
Course Title	Intelligent Systems
Semester/Year	Winter 2026
Instructor	Dr. Mohammed Saif
Section Number	12

Lab 2

Report Title	Lab 2
Submission Date	03/03/2026
Due Date	03/06/2026

Student Name	Student ID	Signature*
Habiba Nagy	XXXX33261	H.N

(Note: Remove the first 4 digits from your student ID)

TA Name		Signature*
Erfan Shahab		Erfan Shahab

**By signing above, you attest that you have contributed to this submission and confirm that all work you have contributed to this submission is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: http://www.ryerson.ca/senate/policies/pol60.pdf.*

1.0 Introduction

Linear classification is one of the simplest and most widely used approaches for separating data into categories. In this lab, a linear classifier is trained using a linear discriminant function, where the decision boundary is a straight line in a two-dimensional feature space. The focus is not only on whether the classifier can separate the classes, but also on how the learning process behaves when trained using gradient descent with the perceptron criterion function.

Using the Iris dataset, the classifier is evaluated on two binary problems (Setosa vs Versicolour and Versicolour vs Virginica) based on two selected features: sepal width and petal length. The lab explores how changing the training/testing split affects generalization, how hyperparameters such as learning rate, threshold, and initial weights influence convergence, and how the criterion function evolves over iterations. Performance is measured using classification accuracy on a held-out test set, and the results are summarized with plots of the data and the learned decision boundaries.

2.0 Dataset and Feature Selection

The Iris dataset contains 150 samples across three classes. For consistency and visualization, only two features were used:

- x_1: sepal width
- x_2: petal length

Two binary classification problems were evaluated:

- Setosa (A) vs Versicolour (B)
- Versicolour (B) vs Virginica (C)

3.0 Method

3.1 Linear Discriminant Function

Each sample is represented using an augmented feature vector:

$$y = \begin{bmatrix} 1 \\ x_1 \\ x_2 \end{bmatrix}$$

and the discriminant function:

$$g(x) = a^T y$$

Classification rule:

- if $g(x) > 0$, assign to the first class (e.g., A or B)
- if $g(x) < 0$, assign to the second class (e.g., B or C)

3.2 Perceptron Criterion and Gradient Descent

Misclassified samples are those with:

$$a^T y \leq 0$$

The perceptron criterion function is defined over the misclassified set:

$$J_p(a) = \sum_{y \epsilon Y(a)} (- a^T y)$$

and its gradient is:

$$\nabla J_p(a) = \sum_{y \epsilon Y(a)} (- y)$$

Weights are updated using gradient descent:

$$\alpha(k + 1) = \alpha(k) - \eta(k) \nabla J_p(a)$$

3.3 Experimental Settings

Unless otherwise stated, the following lab settings were used:

- learning rate: $\eta = 0.01$
- threshold: $\theta = 0$
- initial weights: $a(0) = [0, 0, 1]$
- maximum iterations: 300

Two different data splits were tested for each class pair:

- 30% training / 70% testing
- 70% training / 30% testing

4. Results

4.1 A vs B (Setosa vs Versicolour)

Case 1: 30% Training / 70% Testing

- Final weights: $a = [a0, a1, a2] = [0.01 \quad 0.57 \quad -0.989]$

- Iterations used: 9

- Test accuracy: 100%

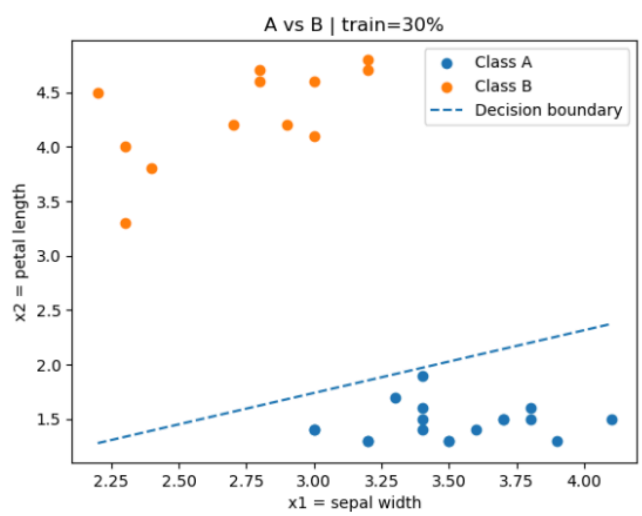

Figure 1: Training data scatter plot with Decision boundary over training data (A vs B)

Case 2: 70% Training / 30% Testing

- Final weights: $a = [0.26 \;\; 1.605 \; - \; 1.632]$
- Iterations used: 7
- Test accuracy: 100**%**

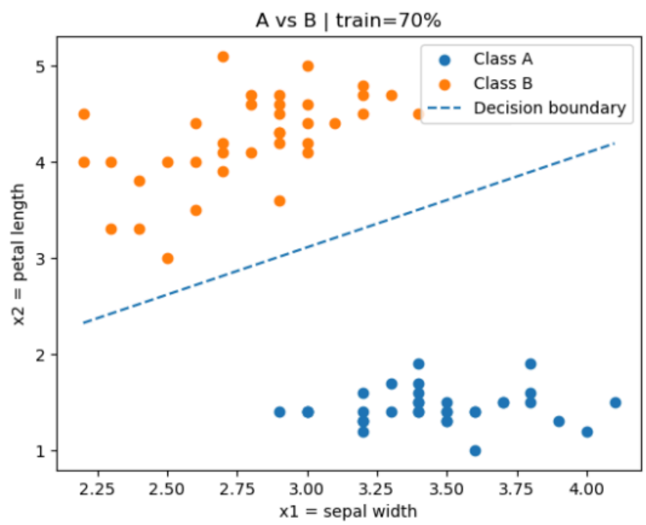

Figure 2: Training data scatter plot with Decision boundary over training data (A vs B)

4.2 B vs C (Versicolour vs Virginica)

Case 1: 30% Training / 70% Testing

- Final weights: $a = [2.83 \;\; 3.673 \; - \; 2.672]$
- Iterations used: 300
- Test accuracy: 82.86**%**

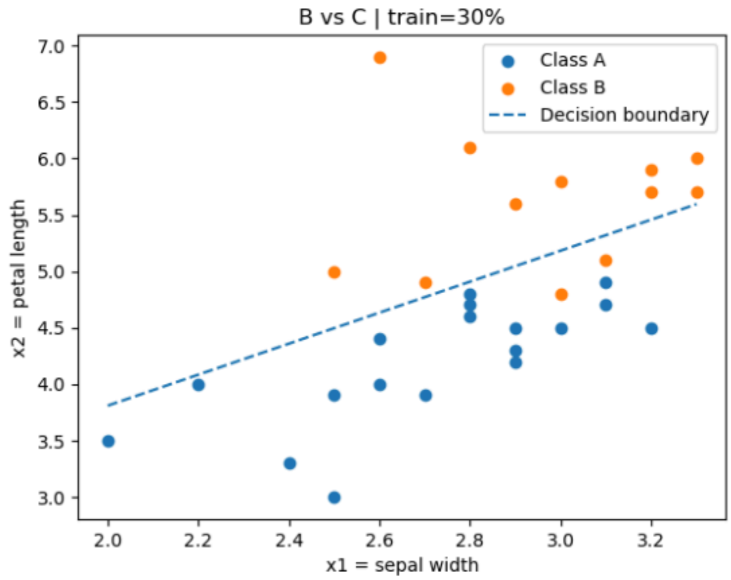

Figure 3: Training data scatter plot with Decision boundary over training data (B vs C)

Case 2: 70% Training / 30% Testing

- Final weights: $a = [5.59 \quad 7.474 \quad -5.324]$
- Iterations used: 300
- Test accuracy: 73.33%

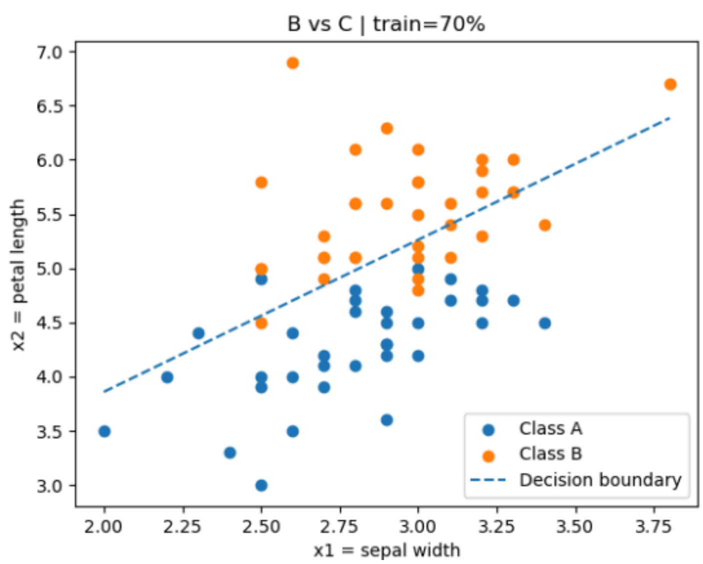

Figure 4: Training data scatter plot with Decision boundary over training data (B vs C)

4.3 Step 5: Effect of Learning Rate, Threshold, and Initial Weights

To analyze convergence, training was repeated for multiple configurations:

- Learning rates tested: $\eta = 0.01$ and 0.1
- Initial weights tested: $a(0) \in \{[0\ 0\ 1], [1\ 1\ 1], [-\ 1\ 0\ 1]\}$

For each configuration, an iteration table was produced with:

1. iteration number k
2. criterion value $J_p(a)$
3. gradient $\nabla J_p(a)$
4. current weight vector $a(k)$

First Test :

$\eta = 0.01$ and $a(0) = [0\ 0\ 1]$

```
final a = [ 0.26   1.605 -1.632]
iterations used = 7
   k     Jp(a)                                            grad  \
0  1  152.6000    [36.0, 100.50000000000001, 152.59999999999997]
1  2  156.1062              [-34.0, -117.4, -49.199999999999996]
2  3   11.0761    [36.0, 100.50000000000001, 152.59999999999997]
3  4  187.8184              [-34.0, -117.4, -49.199999999999996]
4  5   14.3636  [-32.0, -109.80000000000001, -46.99999999999999]
5  6   63.1432    [36.0, 100.50000000000001, 152.59999999999997]
6  7   56.6214              [-34.0, -117.4, -49.199999999999996]

                                                 a  misclassified
0                                  [0.0, 0.0, 1.0]             36
1  [-0.36, -1.0050000000000001, -0.5259999999999998]           34
2  [-0.019999999999999962, 0.16900000000000004, -...           36
3  [-0.37999999999999995, -0.8360000000000001, -1...           34
4  [-0.039999999999999925, 0.3380000000000001, -1...          32
5  [0.2800000000000001, 1.4360000000000002, -0.59...           36
6  [-0.0799999999999999, 0.43100000000000005, -2....           34
```

Figure 5.0: Data for first test

Second Test :

$\eta = 0.01$ and $a(0) = [1\ 1\ 1]$

```
final a = [ 0.78   0.833 -1.834]
iterations used = 6
   k    Jp(a)                                              grad  \
0  1  289.1000     [36.0, 100.50000000000001, 152.59999999999997]
1  2    4.8022  [-33.0, -113.80000000000001, -48.19999999999996]
2  3  142.0721     [36.0, 100.50000000000001, 152.59999999999997]
3  4   41.4768               [-34.0, -117.4, -49.19999999999996]
4  5    6.3090                                [20.0, 57.4, 81.1]
5  6    0.4955                               [-3.0, -10.5, -5.5]

                                                   a  misclassified
0                                    [1.0, 1.0, 1.0]             36
1  [0.64, -0.005000000000001155, -0.525999999999...             33
2                [0.97, 1.133, -0.04399999999999982]             36
3    [0.61, 0.1279999999999999, -1.5699999999999996]             34
4                 [0.95, 1.302, -1.0779999999999996]             20
5    [0.75, 0.7280000000000001, -1.8889999999999996]              3
```

Figure 6.0: Data for second test

Third Test :

$\eta = 0.01$ and $a(0) = [-\ 1\ 0\ 1]$

```
final a = [-0.71   1.717 -1.6  ]
iterations used = 7
   k    Jp(a)                                              grad  \
0  1  116.6000   [35.0, 96.90000000000002, 151.59999999999997]
1  2  185.0478              [-34.0, -117.4, -49.19999999999996]
2  3   11.4538              [-34.0, -117.4, -49.19999999999996]
3  4  185.8863  [36.0, 100.50000000000001, 152.59999999999997]
4  5   43.1660              [-34.0, -117.4, -49.19999999999996]
5  6   44.3624  [36.0, 100.50000000000001, 152.59999999999997]
6  7   74.8782              [-34.0, -117.4, -49.19999999999996]

                                                   a  misclassified
0                                   [-1.0, 0.0, 1.0]             37
1  [-1.35, -0.9690000000000002, -0.5159999999999998]             34
2  [-1.01, 0.2049999999999996, -0.0239999999999998]             34
3   [-0.6699999999999999, 1.379, 0.4680000000000002]             36
4  [-1.0299999999999998, 0.3739999999999999, -1.0...             34
5  [-0.6899999999999997, 1.548, -0.5659999999999996]             36
6  [-1.0499999999999998, 0.5429999999999999, -2.0...             34
```

Figure 7.0: Data for third test

Fourth Test :

$\eta = 0.1$ and $a(0) = [0\ 0\ 1]$

```
final a = [  3.    15.12 -14.76]
iterations used = 5
   k     Jp(a)                                               grad  \
0  1   152.600  [36.0, 100.50000000000001, 152.59999999999997]
1  2  2003.862            [-34.0, -117.4, -49.199999999999996]
2  3   267.922            [-34.0, -117.4, -49.199999999999996]
3  4   790.423  [36.0, 100.50000000000001, 152.59999999999997]
4  5   585.044            [-34.0, -117.4, -49.199999999999996]

                                                 a  misclassified
0                                  [0.0, 0.0, 1.0]             36
1   [-3.6, -10.050000000000002, -14.259999999999998]           34
2  [-0.19999999999999973, 1.6899999999999995, -9....           34
3  [3.2000000000000006, 13.430000000000001, -4.41...           36
4  [-0.39999999999999947, 3.379999999999999, -19....           34
```

Figure 8.0: Data for fourth test

Fifth Test :

$\eta = 0.1$ and $a(0) = [1\ 1\ 1]$

```
final a = [  3.8   15.36 -14.98]
iterations used = 5
   k     Jp(a)                                                 grad  \
0  1   289.100    [36.0, 100.50000000000001, 152.59999999999997]
1  2  1852.462              [-34.0, -117.4, -49.199999999999996]
2  3   118.018  [-32.0, -109.80000000000001, -46.99999999999999]
3  4   809.771    [36.0, 100.50000000000001, 152.59999999999997]
4  5   540.492              [-34.0, -117.4, -49.199999999999996]

                                                 a  misclassified
0                                  [1.0, 1.0, 1.0]             36
1    [-2.6, -9.050000000000002, -14.259999999999998]           34
2  [0.8000000000000003, 2.6899999999999995, -9.33...           32
3     [4.0, 13.670000000000002, -4.639999999999999]            36
4    [0.3999999999999999, 3.619999999999999, -19.9]            34
```

Figure 8.0: Data for fifth test

Sixth Test :

$\eta = 0.1$ and $a(0) = [-1\ 0\ 1]$

```
final a = [  2.1   15.48 -14.66]
iterations used = 5
   k    Jp(a)                                          grad  \
0  1   116.600   [35.0, 96.90000000000002, 151.59999999999997]
1  2  1987.278           [-34.0, -117.4, -49.199999999999996]
2  3   251.338           [-34.0, -117.4, -49.199999999999996]
3  4   809.463  [36.0, 100.50000000000001, 152.59999999999997]
4  5   568.460           [-34.0, -117.4, -49.199999999999996]

                                                   a  misclassified
0                                   [-1.0, 0.0, 1.0]             37
1    [-4.5, -9.690000000000003, -14.159999999999997]             34
2  [-1.0999999999999996, 2.049999999999999, -9.23...             34
3  [2.3000000000000007, 13.790000000000001, -4.31...             36
4  [-1.2999999999999994. 3.7399999999999984. -19....             34
```

Figure 8.0: Data for sixth test

5. Discussion

(a) Effect of different training/testing sizes

Overall, using 70% of the data for training tended to produce a more reliable decision boundary. With a larger training set, the classifier sees more examples and the learned boundary is usually less sensitive to which samples happened to be selected. In contrast, with only 30% training, results can vary more depending on the random split, which can lead to a noticeable change in accuracy and sometimes slower or less stable convergence.

In this lab, the observed accuracies were:

- A vs B: 30/70 → 100%, 70/30 → 100%
- B vs C: 30/70 → 82.86%, 70/30 → 73.33%

Based on these results, it can be observed how with A vs B there was no accuracy change between the different testing sizes as they both find a boundary with 100% accuracy. However, for B vs C the results show that having 70% as the training set gives a higher accuracy, which agrees with the statement above.

(b) Effect of different learning rates, thresholds, and initial weights

The learning rate had the clearest effect on training behavior. A smaller learning rate generally produced steadier progress but required more iterations. A larger learning rate often reduced the number of iterations.

The initial weight vector also affected how quickly the algorithm converged. Some initializations started closer to a reasonable boundary and converged quickly, while others required many updates before reaching a stable separating line.

(c) Behavior of the criterion function over iterations

The perceptron criterion $J_p(a)$ is computed using only the misclassified samples, so its value is closely tied to how many points are currently on the wrong side of the boundary and how far they are from being correctly classified. In the typical case, as training progresses and misclassifications decrease, $J_p(a)$ trends downward.

(d) Classification accuracy achieved

Final classification accuracy was computed on the held-out test set for each split and class pair. As expected, B vs C produced lower accuracy than A vs B because Versicolour and Virginica tend to overlap more in this 2D feature space, making them harder to separate with a single straight line.

6.0 Conclusion

In this lab, a linear discriminant classifier was trained using gradient descent with the perceptron criterion on two Iris classification tasks using sepal width and petal length. For A vs B, the data were clearly linearly separable in the selected feature space, and the algorithm converged quickly while achieving 100% test accuracy for both the 30/70 and 70/30 splits.

For B vs C, the classes showed noticeable overlap, which made the problem more difficult for a single linear boundary. As a result, the algorithm did not fully converge within the 300-iteration limit in either split, and the achieved test accuracy was lower. These results highlight that classification performance depends strongly on how well the chosen features separate the classes.

Finally, the Step 5 experiments showed that learning behavior is sensitive to hyperparameters: changing the learning rate and initial weights affected the convergence speed and the stability of the updates, which was reflected in the iteration tables and plots. Overall, the lab demonstrates that the perceptron-criterion gradient descent approach can be highly effective for linearly separable data, but its performance is limited when the classes are not well-separated by a linear decision boundary.